



Nuclei™ Tee Architecture

Nuclei

All rights reserved by Nucleisys, Inc.

Contents

1	Copyright Notice	1
2	Contact Information	2
3	Revision History	5
4	TEE Introduction	6
5	TEE Related CSRs	7
5.1	Added CSRs List For The TEE	7
5.2	RISC-V Standard CSR For TEE	8
5.2.1	sstatus	8
5.2.2	sie	9
5.2.3	stvec	9
5.2.4	scounteren	9
5.2.5	stvt	10
5.2.6	sscratch	10
5.2.7	sepc	10
5.2.8	scause	11
5.2.9	stval	11
5.2.10	sip	12
5.2.11	snxti	12
5.2.12	sintstatus	12
5.2.13	sscratchcsrw	13
5.2.14	sscratchcswl	13
5.2.15	satp	14
5.2.16	medeleg	14
5.2.17	mideleg	15
5.3	Nuclei Customized CSR For TEE	15
5.3.1	S-mode PMP CSRs	15
5.3.2	S-mode Interrupt/Exception and ECLIC CSRs	15
5.3.2.1	jalsnxti	15
5.3.2.2	stvt2	15
5.3.2.3	sdcause	16
5.3.2.4	pushscause	17
5.3.2.5	pushsepc	18

6	TEE Interrupt Operation	19
6.1	ECLIC Memory Map	19
6.1.1	M-mode ECLIC Region	20
6.1.2	S-mode ECLIC Region	20
6.2	ECLIC Modified Memory Mapped Registers	20
6.2.1	cliccfg	20
6.2.2	minthresh	20
6.2.3	sintthresh	21
6.2.4	clicintattr[i]	21
6.3	ECLIC Interrupt Arbitration	22
6.4	Supervisor-Level Interrupt Flow	22
6.4.1	Enter a Supervisor-Level Interrupt Handler	22
6.4.1.1	Jump to a New PC To Execute	23
6.4.1.2	Update CSR <i>sepc</i>	23
6.4.1.3	Update CSR <i>scause</i>	23
6.4.1.4	Update CSR <i>sstatus</i>	24
6.4.1.5	Update CSR <i>sintstatus</i>	24
6.4.1.6	Privilege Mode Changed to Supervisor Mode	24
6.4.2	Return from a Supervisor-Level Interrupt Handler	24
6.4.2.1	Jump to a New PC To Execute	25
6.4.2.2	Update CSR <i>sstatus</i>	25
6.4.2.3	Update CSR <i>scause</i>	25
6.4.2.4	Update CSR <i>sintstatus</i>	26
6.4.2.5	Privilege Mode changed to the Previous Privilege Mode	26
6.4.3	Supervisor-Level Non-vector Mode	26
6.4.4	Supervisor-Level Vector Mode	26
6.5	Nesting between Privilege Modes	27
7	TEE Exception Operation	28
7.1	TEE Exception Mask	28
7.2	TEE Exception Priority	28
7.3	S-mode Exception Taken	28
7.4	S-mode Exception Return	29
7.5	S-mode Exception Nesting	29
8	TEE Low-Power Mechanism	30
8.1	TEE WFI Mechanism	30
8.2	TEE WFE Mechanism	31
9	TEE Physical Memory Protection Mechanism	32
9.1	TEE PMP Mechanism	32
9.2	TEE sPMP Mechanism	32
9.2.1	sPMP CSRs Introduction	32
9.2.1.1	sPMP CSRs List	32
9.2.1.2	spmpcfg<x>	33
9.2.1.3	spmpaddr<x>	34
9.2.2	sPMP Locking and Privilege Mode	34
9.2.3	sPMP Exception	35
9.2.4	sPMP Priority and Matching Logic	35
9.2.5	SMAP and SMEP with sPMP	35
10	TEE Configuration	36
11	Appendix	37

Copyright Notice

Copyright© 2018-2022 Nuclei System Technology. All rights reserved.

Nuclei are trademarks owned by Nuclei System Technology. All other trademarks used herein are the property of their respective owners.

The product described herein is subject to continuous development and improvement; information herein is given by Nuclei in good faith but without warranties.

This document is intended only to assist the reader in the use of the product. Nuclei System Technology shall not be liable for any loss or damage arising from the use of any information in this document, or any incorrect use of the product.

Contact Information

Should you have any problems with the information contained herein or any suggestions, please contact Nuclei System Technology by email support@nucleisys.com, or visit “Nuclei User Center” website <http://user.nucleisys.com> for supports or online discussion.

List of Figures

5.1	sstatus register	8
5.2	mstatus register	9
5.3	stvec register	9
5.4	scounter register	9
5.5	stvt register	10
5.6	sscratch register	10
5.7	sepc register	10
5.8	scause register	11
5.9	stval register	11
5.10	snxti register	12
5.11	sintstatus register	12
5.12	mintstatus register	12
5.13	sscratchcsw register	13
5.14	sscratchcswl register	13
5.15	Medeleg register	14
5.16	jalsnxti register	15
5.17	stvt2 register	15
5.18	pushscause register	17
5.19	pushsepc register	18
6.1	Modified arbitration scheme	22
6.2	General flow of entering a supervisor-level interrupt handler	23
6.3	General flow of returning from an supervisor-level interrupt handler	25
9.1	RV32 smpcfg<x> layout	33
9.2	RV64 smpcfg<x> layout	33
9.3	RV32 smpaddr<x> format	34
9.4	RV64 smpaddr<x> format	34

List of Tables

5.1	Added CSRs for the TEE	7
5.2	Alignment of stvt base address	10
5.3	scause register description	11
5.4	Medeleg register description	14
5.5	stvt2 register description	16
5.6	sdcause register description	16
6.1	ECLIC Modified Memory Map	19
6.2	cliccfg bit assignments	20
6.3	mintthresh bit assignments	21
6.4	sinthresh bit assignments	21
6.5	clicintattr[i] bit assignments	21
9.1	sPMP CSRs	32
9.2	spmp<x>cfg format	34

Revision History

Rev.	Revision Date	Revised Content
1.0.0	2019/11/12	1. Initial Release.
1.0.1	2021/02/01	1. Add sdcause in 2.2. 2. Modify some typos related with user mode and supervisor mode.

TEE Introduction

Nuclei provides the TEE (Trusted Execution Environment) to achieve better isolation for implementing supervisor-level interrupt/exception handling and sPMP.

With TEE, it is flexible to isolate machine-level interrupt/exception from lower privileged-level interrupt/exception.

When the TEE is configured, and the outer execution environment has delegated specified interrupts and exceptions to supervisor-level, then hardware can transfer control directly to a supervisor-level trap handler without invoking the outer execution environment. Since then, users have more flexibility to define the behavior of hardware when an interrupt or exception is taken. Users can delegate the specified exceptions through the TEE. As for interrupts, with the TEE, users can delegate the designated interrupts only when ECLIC is configured, please refer ECLIC register *clicintattr* in later chapter. The NMI (Non-maskable-interrupt) cannot be trapped to the supervisor-mode or user-mode for any configuration.

Moreover, with TEE, it enables S-mode OS to limit the physical addresses accessible by U-mode software.

When the TEE is configured, sPMP entries can be set to achieve better isolation between OS (running on S-mode) from user software (running on U-mode), and to achieve better scalability of PMP-based TEE/enclave with smaller TCB (Trusted Computing Base).

TEE Related CSRs

5.1 Added CSRs List For The TEE

Based on the basic CSRs, the TEE has added some CSRs as shown in following table. They can be classified to following as the following five categories:

- RISC-V Standard S Mode CSRs:

status, sie, stvec, scouteren, sscratch, spec, scause, stval, sip, satp

- RISC-V Standard M Mode CSRs for S-mode and trap:

medeleg, mideleg

- Nuclei customized CSRS for S-mode PMP:

smpcfg<x>, smpaddr<x>

- RISC-V CLIC Draft Extended for S-mode:

stvt, snxti, sintstauts, stvt2

- Nuclei customized S-mode CSRs for interrupt and exception:

jalsnxti, sscratchcsw, sscratchcswl, pushscause, pushsepc, sdcause

Table 5.1: Added CSRs for the TEE

CSR Type	Number	Privilege	Name	Description
RISC-V Standard CSR	0x100	SRW	sstatus	Supervisor status register
	0x104	SRW	sie	Supervisor interrupt-enable register
	0x105	SRW	stvec	Supervisor trap handler base address
	0x106	SRW	scounteren	Supervisor counter enable
	0x140	SRW	sscratch	Scratch register for supervisor trap handlers
	0x141	SRW	sepc	Supervisor exception program counter
	0x142	SRW	scause	Supervisor trap cause
	0x143	SRW	stval	Supervisor trap value register
	0x144	SRW	sip	Supervisor interrupt pending
	0x180	SRW	satp	Supervisor address translation and protection

continues on next page

Table 5.1 – continued from previous page

CSR Type	Number	Privilege	Name	Description
	0x302	MRW	medeleg	Machine exception delegation register
	0x303	MRW	mideleg	Machine interrupt delegation register
Nuclei Customized CSRs	0x1A0	SRW	spmpcfg0	Supervisor physical memory protection configuration
	0x1A1	SRW	spmpcfg1	Supervisor physical memory protection configuration
	0x1A2	SRW	spmpcfg2	Supervisor physical memory protection configuration
	0x1A3	SRW	spmpcfg3	Supervisor physical memory protection configuration
	0x1B0	SRW	spmpaddr0	Supervisor physical memory protection address register
	0x1B1	SRW	spmpaddr1	Supervisor physical memory protection address register
	...	SRW	...	...
	(0x1B0+n)	SRW	spmpaddrn	Supervisor physical memory protection address register
	0x107	SRW	stvt	Supervisor Trap-handler vector table base address
	0x145	SRW	snxti	Supervisor interrupt handler address and enable modifier
	0x146	SRW	sintstatus	Supervisor current interrupt level
	0x148	SRW	sscratchcs	Scratch swap register for multiple privilege modes
	0x149	SRW	sscratchcs1	Scratch swap register for supervisor interrupt levels
	0x947	SRW	jalsnxti	Jumping to next supervisor interrupt handler address and interrupt-enable register
	0x948	SRW	stvt2	ECLIC non-vectored supervisor interrupt handler address register
	0x949	SRW	pushsc	Push sc to stack
	0x94a	SRW	pushsepc	Push sepc to stack
	0x9c0	SRW	sdcause	Detail information for mcause

5.2 RISC-V Standard CSR For TEE

5.2.1 sstatus

The *sstatus* register is a SXLEN read/write register the lower 32-bits formatted as shown in *sstatus register* (page 8). The *sstatus* register keeps track of and controls the hart's current operating state.



Fig. 5.1: sstatus register

The supervisor interrupt-enable bit SIE indicates supervisor-level interrupts are disabled when it is clear. The value of SIE is preserved in SPIE when a supervisor-level trap is taken, and the value of SIE is set to zero to provide atomicity

for the supervisor-level trap handler.

The SUM (permit Supervisor User Memory access) bit modifies the privilege with which S-mode loads and stores access user mode memory, see *SMAP and SMEP with sPMP* (page 35) for more details about SUM bit.

The SRET instruction is used to return from traps in S-mode, an instruction unique to S-mode. SRET sets PC to sepc, restores interrupt-enable setting by copying SPIE to SIE, and sets SPIE bit.

When the TEE is configured, the *mstatus* register mirrors the SIE and SPIE fields in bit 1 and bit 5, the same location with *sstatus* as shown in *mstatus register* (page 9).

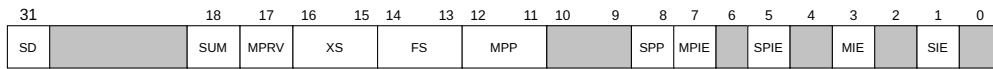


Fig. 5.2: mstatus register

5.2.2 sie

The *sie* register has **NO** effect when interrupt handling mode is ECLIC, and return data are all zeros while reading the register.

5.2.3 stvec

The *stvec* register is a SXLEN read/write register the lower 32-bits formatted as shown in *stvec register* (page 9). It functions in an analogous way to the *mtvec* register defined in M-mode.

The *stvec* register holds S-mode exception and interrupt configuration, consisting of exception and CLIC non-vector base address (BASE).

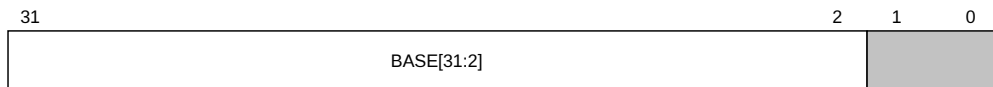


Fig. 5.3: stvec register

5.2.4 scounteren

The counter-enable register *scounteren* controls the availability of the hardware performance monitoring counters to U-mode.

When the CY, TM or IR bit in the *scounteren* register is clear, attempts to read the cycle, time or instret register while executing in U-mode will cause an illegal instruction exception. When one of these bits is set, access to the corresponding register is permitted.

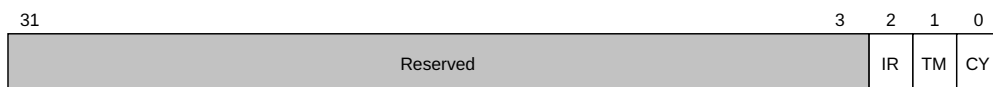


Fig. 5.4: scounter register

5.2.5 stvt

The *stvt* register is a SXLEN read/write register the lower 32-bits formatted as shown in *stvt register* (page 10). It functions in an analogous way to the *mtvt* register defined in M-mode.



Fig. 5.5: stvt register

The *stvt* register holds the base address of ECLIC S-mode vector interrupts, and the base address is aligned at least 64-byte boundary. In order to improve the performance and reduce the gate count, the alignment of the base address in *stvt* is determined by the actual number of interrupts, which is shown in *Alignment of stvt base address* (page 10).

Table 5.2: Alignment of stvt base address

interrupt number	alignment
0 to 16	64-byte
17 to 32	128-byte
33 to 64	256-byte
65 to 128	512-byte
129 to 256	1KB
257 to 512	2KB
513 to 1024	4KB

5.2.6 sscratch

The *sscratch* register is a SXLEN read/write register dedicated for use by supervisor mode. Typically, it is used to hold a pointer to a user mode hart-local context space and swapped with a supervisor register upon entry to a S-mode trap handler.



Fig. 5.6: sscratch register

5.2.7 sepc

The *sepc* register is a SXLEN read/write register the lower 32-bits formatted as shown in *sepc register* (page 10). It functions in an analogous way to the *mepc* register defined in M-mode.

When an exception or interrupt is taken into S-mode, *sepc* is written with the virtual address of the instruction that encountered the exception or interrupt. Otherwise, *sepc* is never written by the implementation, though it may be explicitly written by software.

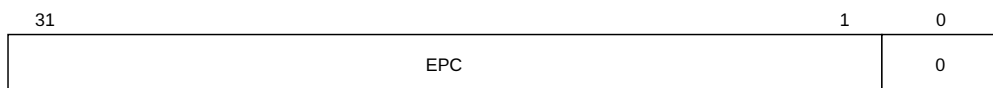


Fig. 5.7: sepc register

5.2.8 scause

The *scause* register is a SXLEN read/write register the lower 32-bits formatted as shown in *scause register* (page 11). It functions in an analogous way to the *mcause* register defined in M-mode.

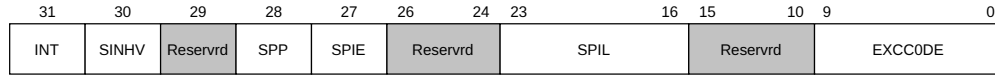


Fig. 5.8: scause register

Table 5.3: scause register description

Field Name	Bit	Description
INT	31	The bit is set if the trap was caused by an interrupt.
SINHV	30	The bit indicates that the interrupt is reading the interrupt vector table.
Reserved	29	0
SPP	28	The bit indicates privilege mode before S-mode exception or interrupt is taken. It mirrors sstatus.spp
SPIE	27	The bit holds the SIE value before S-mode exception or interrupt is taken. It mirrors sstatus.spie
Reserved	26:24	0
SPIL	23:16	The bits hold the interrupt level before S-mode interrupt is taken.
Reserved	15:10	0
EXCCODE	9:0	The bits hold the code of last exception or interrupt

5.2.9 stval

The *stval* register is a SXLEN read/write register the lower 32-bits formatted as shown in *stval register* (page 11). When a trap is taken into S-mode, *stval* is written with exception-specific information to assist software in handling the trap. Otherwise, *stval* is never written by the implementation, though it may be explicitly written by software.

When a hardware breakpoint is triggered, or an instruction-fetch, load, or store address-misaligned, access exception occurs, *stval* is written with the faulting effective address.

On an illegal instruction trap, *stval* is written with the faulting instruction code.

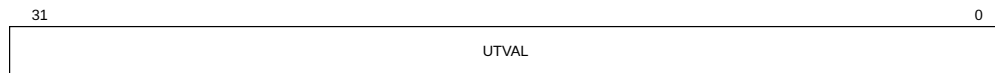


Fig. 5.9: stval register

5.2.10 sip

The *sip* register has **NO** effect when interrupt handling mode is ECLIC, and return data are all zeros while reading the register.

5.2.11 snxti

The *snxti* register is a SXLEN read/write register the lower 32-bits formatted as shown in *snxti register* (page 12). It functions in an analogous way to the *mnxti* register defined in M-mode.

The *snxti* CSR can be used by software to service the next S-mode horizontal or lower level interrupt when it has a higher level than the saved interrupt context (held in *scause.spil*), without incurring the full cost of an interrupt pipeline flush and context save/restore.

Note: If the next interrupt is an M-mode one while the execution privilege mode is S-mode, the processor will take the next interrupt directly in a nested way. In other hand, if the processor is running in M-mode, then a S-mode interrupt will not be taken.

The *snxti* CSR is designed to be accessed using CSRRSI/CSRRCI instructions, where the value read is a pointer to an entry in the S-mode interrupt handler table and the write back updates the S-mode interrupt-enable status.

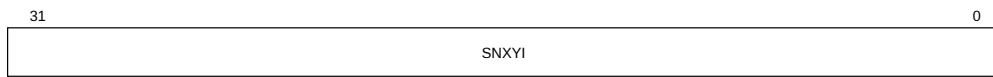


Fig. 5.10: snxti register

5.2.12 sintstatus

The *sintstatus* register is a SXLEN read/write register the lower 32-bits formatted as shown in *sintstatus register* (page 12). It functions in an analogous way to the *mintstatus* register defined in M-mode.

The *sintstatus* register holds the active interrupt level for S-mode. The SIL field is read-only.

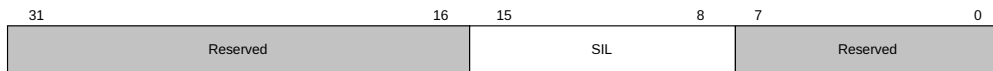


Fig. 5.11: sintstatus register

When the TEE is configured, the *mintstatus* register mirrors the SIL field in bit 8-15, the same location with *sintstatus* as shown in *mintstatus register* (page 12).

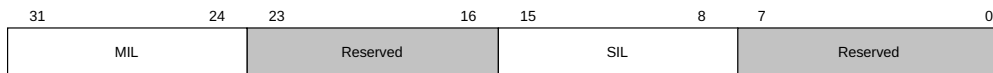


Fig. 5.12: mintstatus register

5.2.13 sscratchcsw

The *sscratchcsw* register is a SXLEN read/write register the lower 32-bits formatted as shown in *sscratchcsw register* (page 13). It functions in an analogous way to the *mscratchcsw* register defined in M-mode.

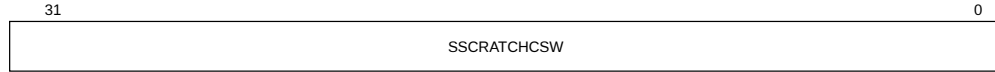


Fig. 5.13: sscratchcsw register

To accelerate interrupt handling with multiple privilege modes, *sscratchcsw* is defined for supervisor mode to support conditional swapping of the *sscratch* register when transitioning between supervisor mode and user mode.

```
csrrw rd, sscratchcsw, rs1

// Pseudocode operation.
if (scause.spp != S-mode) then {
    t = rs1; rd = sscratch; sscratch = t;
} else {
    rd = rs1; // sscratch unchanged.
}

// Usual use: csrrw sp, sscratchcswl, sp
```

5.2.14 sscratchcswl

The *sscratchcswl* register is a SXLEN read/write register the lower 32-bits formatted as shown in *sscratchcswl register* (page 13). It functions in an analogous way to the *mscratchcswl* register defined in M-mode.

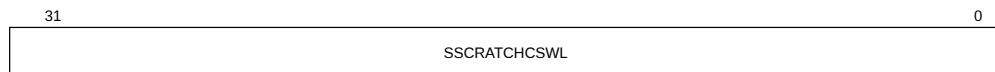


Fig. 5.14: sscratchcswl register

Within U-mode, *sscratchcswl* is useful to separate interrupt handler tasks from application tasks to enhance robustness, reduce space usage, and aid in system debugging. Interrupt handler tasks only have non-zero interrupt levels, while application tasks have an interrupt level of zero.

The *sscratchcswl* CSR is added to support faster swapping of the stack pointer between S-mode interrupt and non-interrupt code running in the same privilege mode.

```
csrrw rd, sscratchcswl, rs1

// Pseudocode operation.
```

(continues on next page)

(continued from previous page)

```

if ( (scause.spil==0) != (sintstatus.sil==0) ) then {
    t = rs1; rd = sscratch; sscratch = t;
} else {
    rd = rs1; // sscratch unchanged.
}

// Usual use: csrrw sp, sscratchcswl, sp

```

5.2.15 satp

The *satp* CSR provides the Supervisor Address Translation and Protection.

5.2.16 medeleg

The *medeleg* register is a SXLEN read/write register the lower 32-bits formatted as shown in *Medeleg register* (page 14). To increase performance, the *medeleg* register contains individual read/write bits to indicate that certain exceptions should be processed directly by a lower privilege level.

By default, all exceptions at any privilege level are handled in machine mode. When N-Extension is configured, setting a bit in *medeleg* will delegate the corresponding exception in S/U-mode to the S-mode exception handler.

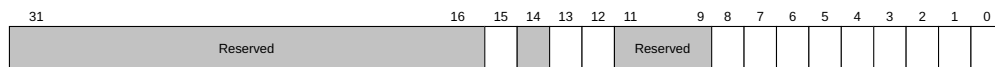


Fig. 5.15: Medeleg register

Table 5.4: Medeleg register description

Field Name	Bits	Description
Reserved	SXLEN-1:16	0
	15	Delegate Store/AMO Page Fault Exception
Reserved	14	0
	13	Delegate Load Page Fault Exception
	12	Delegate Instruction Page Fault Exception
	11	Not used, tie to 0
Reserved	10	0
	9	Not used, tie to 0
	8	Delegate Environment Call from U-Mode
	7	Delegate Store/AMO Access Fault Exception
	6	Delegate Store/AMO Address Misaligned Exception
	5	Delegate Load Access Fault Exception
	4	Delegate Load Access Misaligned Exception
	3	Delegate Breakpoint Exception
	2	Delegate Illegal Instruction Exceptio

continues on next page

Table 5.4 – continued from previous page

Field Name	Bits	Description
	1	Delegate Instruction Access Fault Exception
	0	Delegate Instruction Access Misaligned Exception

5.2.17 mideleg

The *mideleg* register has **NO** effect when interrupt handling mode is ECLIC, and return data are all zeros while reading the register.

5.3 Nuclei Customized CSR For TEE

5.3.1 S-mode PMP CSRs

The *spmpcfg<x>* and *spmpaddr<x>* are detailed described in Section *sPMP CSRs Introduction* (page 32).

5.3.2 S-mode Interrupt/Exception and ECLIC CSRs

5.3.2.1 jalsnxti

The *jalsnxti* register is a SXLEN read/write register the lower 32-bits formatted as shown in *jalsnxti register* (page 15). It functions in an analogous way to the *jalmnxti* register defined in M-mode.

The *jalsnxti* register is designed to speed up the S-mode interrupt handling comparing to using *snxti*. In addition to enabling the interrupt, accessing this CSR by ‘csrrw ra, jalsnxti, ra’ will directly jump to the next S-mode horizontal or lower level interrupt entry address for the same privilege mode, at the same time, save the return address which is the pc of the executing instruction, to the ra register. For more details please refer to *Supervisor-Level Non-vector Mode* (page 26).

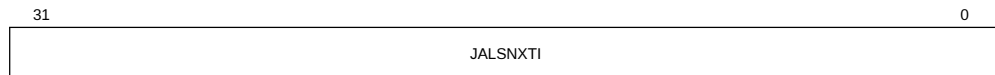


Fig. 5.16: jalsnxti register

5.3.2.2 stvt2

The *stvt2* register is a SXLEN read/write register the lower 32-bits formatted as shown in *stvt2 register* (page 15). It functions in an analogous way to the *mtvt2* register defined in M-mode.

The *stvt2* register holds S-mode ECLIC configuration, consisting of ECLIC non-vector base address(BASE) and ECLIC enable(STVT2EN). The ECLIC non-vector base address is aligned on a 4-byte boundary

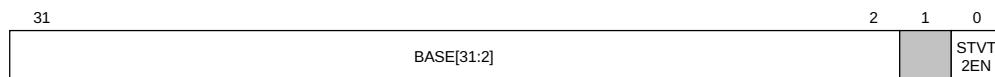


Fig. 5.17: stvt2 register

Table 5.5: stvt2 register description

Field Name	Bits	Description
BASE	SXLEN-1:2	The base address of ECLIC non-vector interrupt in S-mode
Reserved	1	0
STVT2EN	0	Setting this bit will enable ECLIC mode, that means non-vector interrupt base address in S-mode is specified by stvt2 instead of stvec

5.3.2.3 sdcause

Since there might be some exceptions share the same scause.EXCCODE value. To further record the differences, Nuclei processor core customized CSR *sdcause* register to record the detailed information about the exception.

Table 5.6: sdcause register description

Field	Bit	Description
Reserved	SXLEN-1:2	Reserved 0

continues on next page

Table 5.6 – continued from previous page

Field	Bit	Description
sdcause	2:0	Further record the detailed information about the exception. When scause.EXCCODE = 1 (Instruction access fault) • 0: Reserved • 1: PMP permission violation • 2: Bus error • 3-7: Reserved When scause.EXCCODE = 5 (Load access fault) • 0: Reserved • 1: PMP permission violation • 2: Bus error • 3: NICE extended long pipeline instruction return error. Note: although this error ideally is nothing to do with the Load access fault, but they just shared the same scause.EXCCODE to simplify the hardware implementation • 4-7: Reserved When scause.EXCCODE = 7 (Store/AMO access fault) • 0: Reserved • 1: PMP permission violation • 2: Bus error • 3-7: Reserved When scause.EXCCODE = 12 (Instruction page fault) • 0-4: Reserved • 5: Page fault • 6: SPMP permission violation • 7: Reserved When scause.EXCCODE = 13 (Load page fault) • 0-4: Reserved • 5: Page fault • 6: SPMP permission violation • 7: Reserved When scause.EXCCODE = 15 (Store/AMO page fault) • 0-4: Reserved • 5: Page fault • 6: SPMP permission violation • 7: Reserved

5.3.2.4 pushscause

The *pushscause* register is a SXLEN read/write register the lower 32-bits formatted as shown in *pushscause register* (page 17). It functions in an analogous way to the *pushmcause* register defined in M-mode.

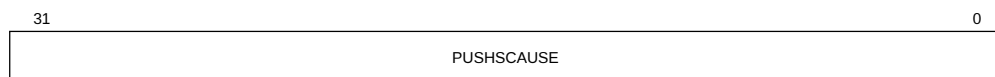


Fig. 5.18: pushscause register

The *pushscause* register is a function CSR and it can't be read or written as a normal CSR. Any CSR instruction except for *crrwi* accessing this register will raise an illegal instruction exception.

The *pushscause* register is designed to speed up the context saving of S-mode interrupt. Using *csrrwi* instruction, the data of *scause* will be pushed to memory address = *sp* + *imm**4.

```
csrrwi x0, pushscause, 2 // push scause to memory address = sp + 8
```

5.3.2.5 pushsepc

The *pushsepc* register is a SXLEN read/write register the lower 32-bits formatted as shown in *pushsepc register* (page 18). It functions in an analogous way to the *pushmepc* register defined in M-mode.

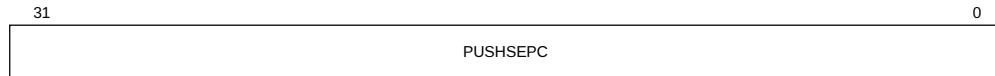


Fig. 5.19: pushsepc register

The *pushsepc* register is a function CSR and it can't be read or written as a normal CSR. Any CSR instruction except for *csrrwi* accessing this register will raise an illegal instruction exception.

The *pushsepc* register is designed to speed up the context saving of S-mode interrupt. Using *csrrwi* instruction, the data of *sepc* will be pushed to memory address = *sp* + *imm**4.

```
csrrwi x0, pushsepc, 2 // push sepc to memory address = sp + 8
```

TEE Interrupt Operation

This chapter describes the behavior of TEE interrupts, especially for supervisor-level interrupts. A supervisor-level interrupt is taken only when the current mode is supervisor mode and the *sstatus.SIE* field is set or when the current mode is user mode. Several main changes in the TEE will be mentioned in the following sections.

6.1 ECLIC Memory Map

The ECLIC memory map has been changed to support the TEE. The ECLIC memory map can support up to 1,024 total interrupt inputs, and an M-mode ECLIC Region and a S-mode ECLIC Region. *ECLIC Modified Memory Map* (page 19) describes the detailed memory map scheme.

Table 6.1: ECLIC Modified Memory Map

Offset	R/W	Name	Width(bits)	Description
0x0000	RW	cliccfg	8	-
0x0004	R	clicinfo	32	-
0x0008	RW	mintthresh	32	-
0x1000+4*i	RW	clicintip[i]	8	M-mode ECLIC Region
0x1001+4*i	RW	clicintie[i]	8	
0x1002+4*i	RW	clicintattr[i]	8	
0x1003+4*i	RW	clicintctl[i]	8	
0x2008	RW	sinthresh	32	-
0x3000+4*i	RW	clicintip[i]	8	S-mode ECLIC Region
0x3001+4*i	RW	clicintie[i]	8	
0x3002+4*i	RW	clicintattr[i]	8	
0x3003+4*i	RW	clicintctl[i]	8	

Note:

- The above “i” indicates the interrupt ID, an interrupt *i* has its own corresponding clicintip[i], clicintie[i], clicintattr[i], and clicintctl[i] registers.
- Aligned byte, half-word, and word accesses to ECLIC registers are supported.
- The above “R” means read-only, and any write to this read-only register will be ignored without bus error.

- If an input i is not present in the hardware, the corresponding `clicintip[i]`, `clicintie[i]`, `clicintctl[i]` memory locations appear hardwired to zero.
-

6.1.1 M-mode ECLIC Region

This region is designed to support M-mode access. If an input i is not present in the hardware, the corresponding `clicintip[i]`, `clicintie[i]`, `clicintctl[i]` memory locations appear hardwired to zero.

6.1.2 S-mode ECLIC Region

Supervisor-mode ECLIC regions only expose interrupts that have been configured to be supervisor-accessible via the M-mode CLIC region. System software must configure PMP and sPMP permissions to make sure this region can only be accessed from appropriate Supervisor-mode codes.

Any interrupt i that is not accessible to S-mode appears as hard-wired zeros in `clicintip[i]`, `clicintie[i]`, and `clicintctl[i]`.

If `clicintattr[i]` is set to S-mode (bit 7 is clear, and bit 6 is set), then interrupt i is visible in the S-mode region except that only the low 6 bits of `clicintattr[i]` can be written via the S-mode memory region.

6.2 ECLIC Modified Memory Mapped Registers

Following only describes the ECLIC modified memory mapped registers.

6.2.1 cliccfg

This `cliccfg` register is a global configuration register. [cliccfg bit assignments](#) (page 20) describes the bit assignments of this register.

Table 6.2: cliccfg bit assignments

Field	Bits	R/W	Reset Value	Description
Reserved	7	R	N/A	Reserved, ties to 0.
nmbits	6:5	R	N/A	nmbits ties to 1, indicates supervisor-level interrupt supporting.
nlbits	4:1	RW	0	nlbits specifies interrupt level and priority.
Reserved	0	R	N/A	Reserved, ties to 1.

6.2.2 mintthresh

The `mintthresh` register holds the current threshold level for each privilege mode (i.e., `mt`, `st`). [mintthresh bit assignments](#) (page 21) describes the bit assignments of this register.

Table 6.3: mintthresh bit assignments

Field	Bits	R/W	Reset Value	Description
mtsh	31:24	RW	0	Interrupt-level threshold for M-mode.
Reserved	23:16	R	N/A	Reserved, ties to 0.
sth	15:8	RW	0	Interrupt-level threshold for S-mode.
Reserved	7:0	R	N/A	Reserved, ties to 0.

6.2.3 sintthresh

The *sintthresh* register holds the current threshold level for Supervisor mode. *sintthresh bit assignments* (page 21) describes the bit assignments of this register.

Table 6.4: sintthresh bit assignments

Field	Bits	R/W	Reset Value	Description
Reserved	31:24	R	N/A	Reserved, ties to 0.
Reserved	23:16	R	N/A	Reserved, ties to 0.
sth	15:8	RW	0	Interrupt-level threshold for S-mode.
Reserved	7:0	R	N/A	Reserved, ties to 0.

Note: here sth is a mirror to mintthresh.sth and will be updated synchronously.

6.2.4 clicintattr[i]

This register specifies various attributes for each interrupt. *clicintattr[i] bit assignments* (page 21) describes the bit assignments of this register.

Table 6.5: clicintattr[i] bit assignments

Field	Bits	R/W	Reset Value	Description
mode	7:6	M-mode: RW S-mode: R	3	Specifies in which privilege mode the interrupt should be taken: 3: Machine Mode 1: Supervisor Mode Note: M-mode can Read/Write this field, but S-mode can only Read this field. So ECLIC with TEE does not reply on CSR <i>mideleg</i> to delegate interrupts. It can only write 1 or 3 to this field, other values are ignored.
Reserved	5:3	R	N/A	Reserved, ties to 0.
trig	2:1	RW	0	Specifies the trigger type and edge polarity for each interrupt input.
shv	0	RW	0	Specifies hardware vectoring or non-vectoring mode.

6.3 ECLIC Interrupt Arbitration

As mentioned above, the *clicintattr[i].mode* field is implemented to specify in which privilege mode the interrupt should be taken. In order to support the TEE, the arbitration logic needs to take the *clicintattr[i].mode* into the consideration. Hence, the winner of the arbitration is determined by interrupts' mode, level and priority. In the modified arbitration logic, privilege mode prioritizes level and priority, and level prioritizes priority. *Modified arbitration scheme* (page 22) shows the modified arbitration scheme. For each interrupt *i*, its privilege mode, level and priority are combined to represent an unsigned integer number. The interrupt that has the greatest number wins the arbitration.

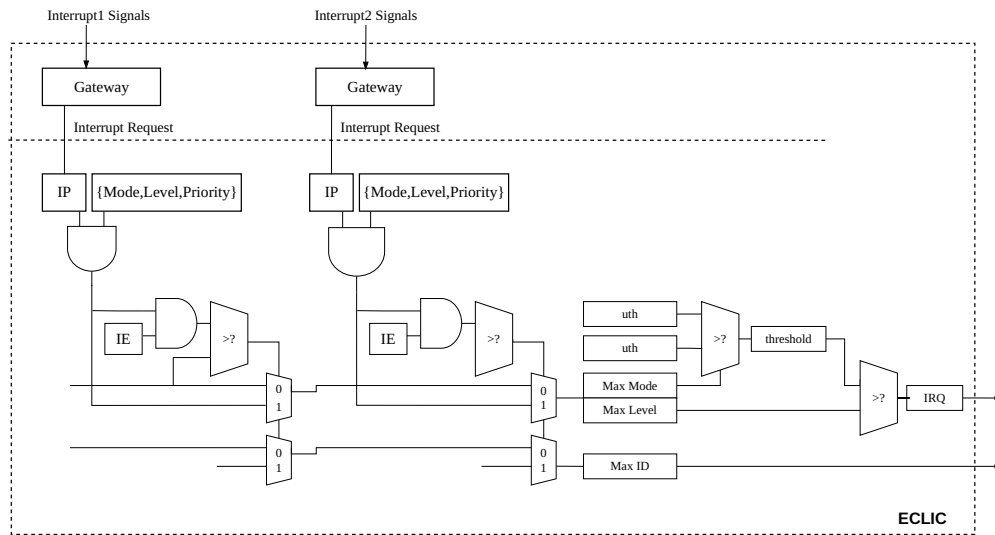


Fig. 6.1: Modified arbitration scheme

6.4 Supervisor-Level Interrupt Flow

This section describes the overall behavior when a supervisor-level interrupt is taken.

6.4.1 Enter a Supervisor-Level Interrupt Handler

If a supervisor-level interrupt wins the arbitration and is taken (it can only happen in supervisor/user mode), the following listed hardware behaviors will happen at the same time:

- Stop executing the current program, and jump to a new PC, then execute.
- Update the following listed CSRs at the same time:
 - sepc*
 - sstatus*
 - scause*
 - sintstatus*
- The privilege mode changes to supervisor mode.
- *General flow of entering a supervisor-level interrupt handler* (page 23) shows the general flow of entering a supervisor-level interrupt handler.

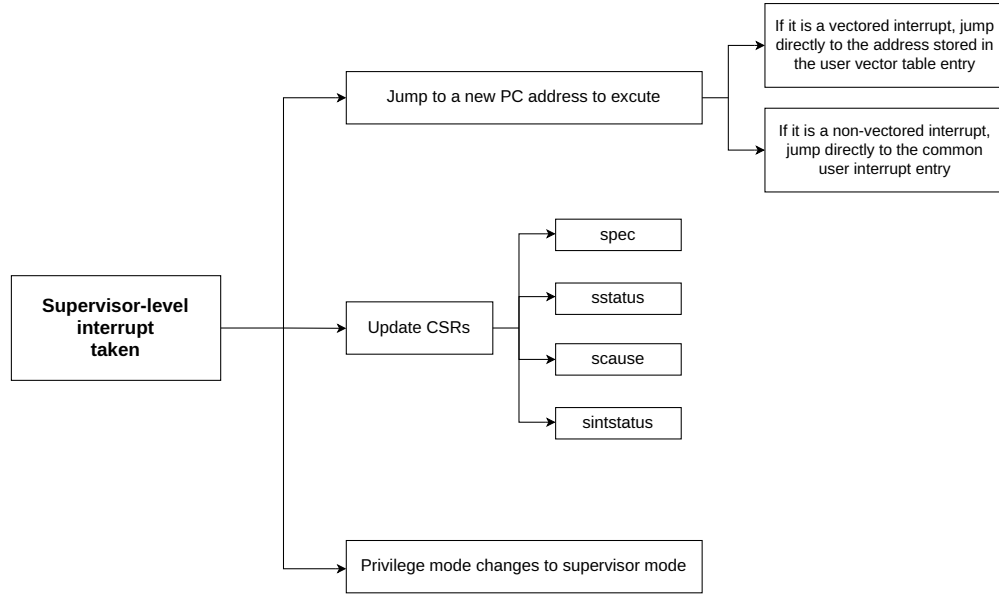


Fig. 6.2: General flow of entering a supervisor-level interrupt handler

6.4.1.1 Jump to a New PC To Execute

Each interrupt can be configured to vectored or non-vectored (through *clicintattr[i].shv*).

- If the interrupt is vectored, the core will jump directly to the address stored in the supervisor vector table entry, which is specified by *stvt*.
- If the interrupt is non-vectored, the core will jump directly to the common supervisor interrupt entry, which is specified by *stvt2*.

6.4.1.2 Update CSR *sepc*

When a supervisor-level interrupt is taken, the *sepc* will be updated to the interrupted PC. In this way, once returning from this supervisor-level interrupt handler, the core can restore the PC from *spec*.

6.4.1.3 Update CSR *scause*

The *scause* will be updated as follows:

- The *scause.EXCCODE* field will be updated to indicate the current interrupt ID.
- The *scause.SPIL* field will be updated to indicate the interrupted interrupt level(ie, *sintstatus.SIL*). In this way, once returning from supervisor-level interrupt handler, *sintstatus.SIL* can restore is interrupted value from *scause.SPIL*.
- If the interrupt is vectored, the *scause.SINHV* field will be updated to indicate its hardware vectoring state.

6.4.1.4 Update CSR *sstatus*

The *sstatus* will be updated as follows:

- The *sstatus*.SPIE field will be updated to the value of *sstatus*.SIE field, while *sstatus*.SIE field will be updated to zero.
- Note: The *scause*.SPIE field mirrors the *sstatus*.SPIE field, and is aliased into *scause* to reduce context save/restore code.

6.4.1.5 Update CSR *sintstatus*

The *sintstatus* will be updated as follows:

- The *sintstatus*.SIL field will be updated to hold the active interrupt level.

6.4.1.6 Privilege Mode Changed to Supervisor Mode

A supervisor-level interrupt can be taken from supervisor/user mode, after being taken the privilege mode will change to supervisor mode.

6.4.2 Return from a Supervisor-Level Interrupt Handler

The regular *sret* instruction is used to return from a supervisor-level interrupt handler. After the execution of *sret*, the following listed hardware behaviors will happen at the same time.

- Stop executing the current program, and jump to a new PC stored in *sepc* to execute.
- Update the following listed CSRs at the same time:

sstatus

scause

sintstatus

- The privilege mode changes to the previous interrupted mode.
- *General flow of returning from an supervisor-level interrupt handler* (page 25) shows the general procedure.

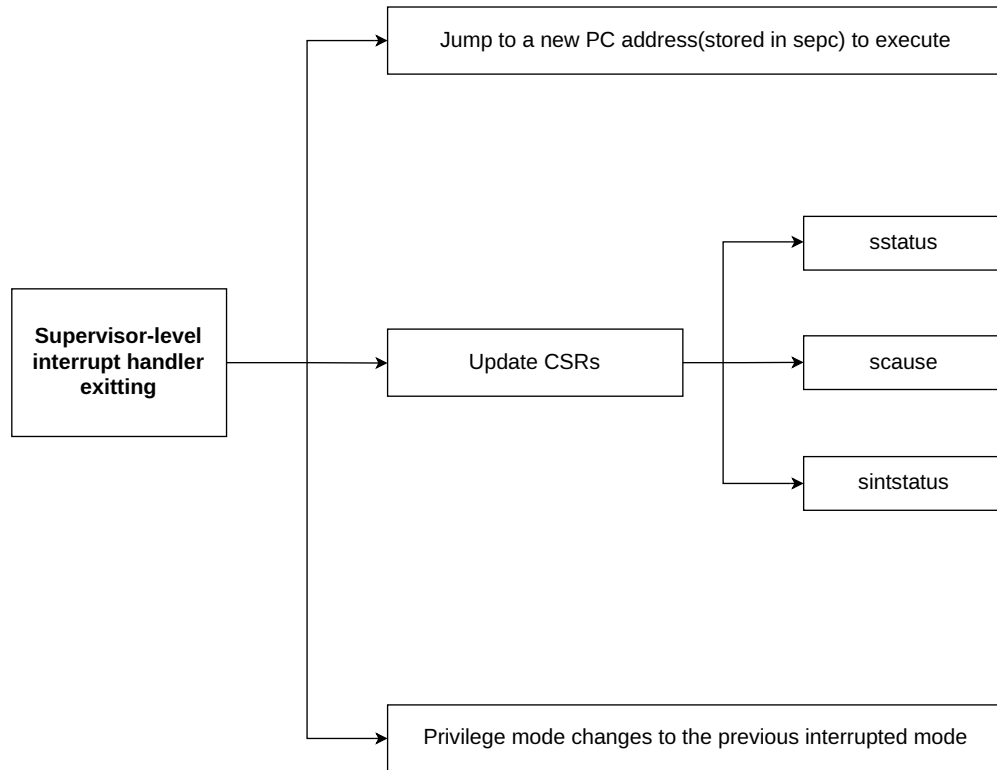


Fig. 6.3: General flow of returning from an supervisor-level interrupt handler

6.4.2.1 Jump to a New PC To Execute

The core will jump to a new address stored in `sepc` to execute, after executing the `sret` instruction to return from a supervisor-level interrupt handler.

6.4.2.2 Update CSR *sstatus*

After the execution of `sret`, the CSR *sstatus* will be updated as follows.

- The *sstatus*.SIE field will be updated to the value of *sstatus*.SPIE field.
- The *sstatus*.SPIE field will be set to 1.

6.4.2.3 Update CSR *scause*

After the execution of `sret`, the CSR *scause* will be updated as follows.

- Note: The *scause*.SPIE field mirrors the *sstatus*.SPIE field, and is aliased into *scause* to reduce context save/restore code.

6.4.2.4 Update CSR *sintstatus*

After the execution of `sret`, the CSR *sintstatus* will be updated as follows.

- The *sintstatus*.SIL field will be updated to the value of the *scause*.SPIEL field.

6.4.2.5 Privilege Mode changed to the Previous Privilege Mode

The privilege mode will be changed to the previous privilege mode (which is stored in *sstatus*.SPP) when returning from this interrupt handler.

6.4.3 Supervisor-Level Non-vector Mode

Supervisor-Level non-vector mode interrupt flow is the same as machine-level non-vector mode flow, except:

- The CSRs that should be saved and restored during interrupt processing are different.
During supervisor-level interrupt processing, *sepc* and *scause* should be saved and restored.
- The instruction used for tail-chaining is different.

For supervisor-level non-vector mode, the instruction “`csrrw ra, jalsnxti, ra`” is used to realize supervisor-level interrupt tail-chaining.

6.4.4 Supervisor-Level Vector Mode

Supervisor-Level vector mode interrupt flow is the same as machine-level vector mode flow, except:

- The CSRs that should be saved and restored during interrupt processing are different.
During supervisor-level interrupt processing, *sepc* and *scause* should be saved and restored.
- The “interrupt attribute” for C is different.

For a supervisor-level vector mode handler, the “interrupt attribute” for C has the following syntax:

```

1 void __attribute__((interrupt("supervisor"))) foo (void)
2
3 {
4
5     extern volatile int INTERRUPT_FLAG;
6
7     INTERRUPT_FLAG = 0;
8
9     extern volatile int COUNTER;
10
11     #ifdef __riscv_atomic
12
13         __atomic_fetch_add (&COUNTER, 1, __ATOMIC_RELAXED);
14
15     #else
16
17         COUNTER++;
18
19     #endif

```

(continues on next page)

(continued from previous page)

20

21

}

6.5 Nesting between Privilege Modes

A machine-level interrupt can pre-empt a supervisor-level interrupt which has already been taken in supervisor mode, while a supervisor-level interrupt can never pre-empt a machine-level interrupt. Besides, a supervisor-level interrupt occurring in machine mode can not be taken.

TEE Exception Operation

This chapter describes the behavior of TEE exception, especially for S-mode exception. The S-mode exception is taken only when the current mode is supervisor mode and the corresponding bit in *medeleg* is set or the current mode is user mode. On the other hand, the *medeleg* register will be ignored when the processor is in machine mode, which means any exception happens in machine mode will never be delegated.

7.1 TEE Exception Mask

For a processor that supports the TEE, unlike interrupts, exceptions can't be masked in any privilege mode at any time. Once there is an exception, the processor will take it in either machine or supervisor mode.

7.2 TEE Exception Priority

The priority of exceptions in one privilege mode is defined by exception code, that is, the exception with smaller code has a higher priority. If a S-mode exception and an M-mode exception occur at the same time in supervisor mode, the M-mode exception has a higher priority.

7.3 S-mode Exception Taken

When entering a S-mode exception, the hardware behavior can be briefly described as follows.

- The PC is changed to the address defined in the *stvec*.
- *scause* is updated to reflect the type of exception.
- *sepc* is updated to save the PC address where the exception happens.
- *stval* is updated to record the memory access address or instruction code.
- *sstatus.sie* is copied to *sstatus.spie* and *sstatus.sie* is cleared.

7.4 S-mode Exception Return

When returning from a S-mode exception trap handler, the hardware behavior can be briefly described as follows.

- The PC is restored from the address in *sepc*.
- *sstatus.sie* is copied to *sstatus.sipe* and *spie* is set.

7.5 S-mode Exception Nesting

Nuclei TEE does not support S-mode exception nesting with another S-mode exception. The behavior is unpredictable.

However, M-mode exception or NMI taken in S-mode exception is controllable, as the S-mode trap CSRs won't be updated.

TEE Low-Power Mechanism

8.1 TEE WFI Mechanism

There are some changes for WFI mechanism in the TEE. Due to the addition of S-mode interrupt, WFI can be divided into U-mode WFI, S-mode WFI and M-mode WFI.

U-mode WFI means that the processor executes wfi instruction in User Mode, then the processor enters the User-Level Low-Power Mode.

- The processor will be waked up if a S-mode interrupt wins the interrupt arbitration, and the processor will be trapped to the corresponding handler no matter *sstatus.sie* is set or not.
- The processor will be waked up if an M-mode interrupt wins the interrupt arbitration, and the processor will be trapped to the corresponding handler no matter *mstatus.mie* is set or not.

S-mode WFI means that the processor executes wfi instruction in Supervisor Mode, then the processor enters the Supervisor-Level Low-Power Mode.

- The processor will be waked up if a S-mode interrupt wins the interrupt arbitration, and whether the processor will resume execution or be trapped to the interrupt handler depends on *sstatus.sie*.
- The processor will be waked up if an M-mode interrupt wins the interrupt arbitration, and the processor will be trapped to the corresponding handler no matter *mstatus.mie* is set or not.

M-mode WFI means that the processor executes wfi instruction in Machine Mode, then the processor enters the Machine-Level Low-Power Mode.

- Any S-mode interrupts cannot win the interrupt arbitration when the processor stays in the Machine-Level Low-Power mode, so the processor will still stay in Low-Power mode when it encounters a S-mode interrupt.
- The processor will be waked up if an M-mode interrupt wins the interrupt arbitration, and whether the processor will resume execution or be trapped to the interrupt handler depends on *mstatus.mie*.

In addition, NMI and debug request can also wake up U-mode WFI, S-mode WFI and M-mode WFI.

8.2 TEE WFE Mechanism

The WFE mechanism for a core with the TEE remains unchanged. No matter what the privilege mode is, setting *wfe* and executing the *wfi* instruction will take the processor to Low-Power mode. The processor could be waked up by event, NMI, and debug request.

TEE Physical Memory Protection Mechanism

In addition to PMP, sPMP mechanism is implemented to achieve better isolation in TEE.

9.1 TEE PMP Mechanism

The PMP mechanism in TEE remains unchanged.

9.2 TEE sPMP Mechanism

The sPMP mechanism provides supervisor-mode control CSRs to allow physical memory access privileges (read, write, execute) to be specified for each physical memory region. The sPMP values are checked after the physical address to be accessed pass PMP checks described in the RISC-V privileged spec.

sPMP checks are applied to all accesses when the hart is running in U-mode, and for load/store when the MPRV bit is set in the *mstatus* CSR and the MPP field in the *mstatus* CSR contains U-mode. Optionally, sPMP checks can also apply to S-mode accesses, in which case the sPMP values are locked to S-mode software, so that S-mode cannot change their values. Unlike PMP registers, sPMP registers can always be modified by M-mode software even when they are locked. sPMP registers can grant permissions to U-mode, which has none by default, and revoke permissions from S-mode, which has full permissions by default.

9.2.1 sPMP CSRs Introduction

9.2.1.1 sPMP CSRs List

Following *sPMP CSRs* (page 32) lists the sPMP CSRs. Like PMP, sPMP entries are described by an 8-bit configuration region and one SXLEN-bit address CSR. Some sPMP settings additionally use the address register associated with the preceding sPMP entry. The number of sPMP entries can vary by implementation, and up to 16 sPMP entries is supported.

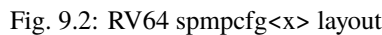
Table 9.1: sPMP CSRs

Number	Privilege	Name	Description
0x1A0	SRW	smpcfg0	Supervisor physical memory protection configuration

continues on next page

Number	Privilege	Name	Description
0x1A1	SRW	smpcfg1	Supervisor physical memory protection configuration (RV32 ONLY)
0x1A2	SRW	smpcfg2	Supervisor physical memory protection configuration
0x1A3	SRW	smpcfg3	Supervisor physical memory protection configuration (RV32 ONLY)
0x1B0	SRW	smpaddr0	Supervisor physical memory protection address register
0x1B1	SRW	smpaddr1	Supervisor physical memory protection address register
...	SRW	...	...
(0x1B0+n)	SRW	smpaddrn	Supervisor physical memory protection address register

The sPMP configuration registers are packed into CSRs in the same way as PMP does. For RV32, four CSRs, `spmpcfg0~spmpcfg3`, hold the configurations `spmp0cfg~spmp15cfg` for the 16 PMP entries, as shown in [RV32 *spmpcfg<x> layout*](#) (page 33). For RV64, `spmpcfg0` and `spmpcfg2` hold the configurations for the 16 PMP entries, as shown in [RV64 *spmpcfg<x> layout*](#) (page 33); `spmpcfg1` and `spmpcfg3` are illegal.



The R, W, and X bits, when set, indicate that the sPMP entry permits read, write and instruction execution, respectively. When one of these bits is clear, the corresponding access type is denied.

The U bit represents that the sPMP entry is for user mode, and will be used to enforce SMAP and SMEP (described in *SMAP and SMEP with sPMP* (page 35)).

The A bits encode the address-matching mode of the associated sPMP entry. The encoding of A field is the same as PMP's.

The remaining L field will be described in the following *sPMP Locking and Privilege Mode* (page 34).

Table 9.2: `spmp<x>cfg` format

Field	Bits	Reset Value	Description
L	7	0	Indicates the sPMP entry is locked.
U	6	0	Indicates the sPMP entry is for user mode.
Reserved	5	N/A	Reserved, ties to 0.
A	4:3	0	Encodes the sPMP entry's address-matching mode.
X	2	0	Indicates the sPMP entry permits execution.
W	1	0	Indicates the sPMP entry permits write.
R	0	0	Indicates the sPMP entry permits read.

9.2.1.3 `spmpaddr<x>`

The sPMP address registers are CSRs named `spmpaddr0`-`spmpaddr15`. Each SPMP address register encodes bits 33-2 of 34-bit physical address for RV32, as shown in *RV32 `spmpaddr<x>` format* (page 34). For RV64, each SPMP address bits may be implemented, and so the `spmpaddr` registers are WARL.

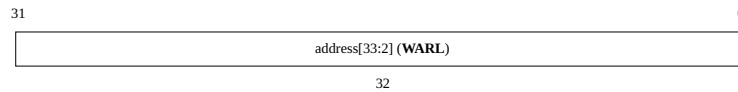


Fig. 9.3: RV32 `spmpaddr<x>` format



Fig. 9.4: RV64 `spmpaddr<x>` format

9.2.2 sPMP Locking and Privilege Mode

The L bit indicates that the sPMP is locked to S-mode, i.e., S-mode writes to the configuration register and associated address registers are ignored. Locked sPMP entries can only be unlocked by M-mode or by a system reset. If sPMP entry *i* is locked, writes to the `spmp<x>cfg` and `mpmpaddr<x>` are ignored. In addition to locking the sPMP entry, the L bit indicates whether the R/W/X permissions are enforced on S-mode accesses. When the L bit is set, these permissions are enforced for both user and supervisor modes (M-mode accesses are not affected). When the L bit is clear, any S-mode access matching the sPMP entry will succeed; the R/W/X permissions apply only to U modes.

9.2.3 sPMP Exception

Failed accesses generate a load, store, or instruction page fault (these exceptions should be delegated to S-mode software when happen).

9.2.4 sPMP Priority and Matching Logic

The sPMP checks only take effect after the memory access passes the PMP permission checks. An M-mode access will not be checked by sPMP property.

Like PMP entries, sPMP entries are also statically prioritized. The lowest-numbered sPMP entry that matches any byte of an access determines whether that access succeeds or fails. The matching sPMP entry must match all bytes of an access, or the access fails, irrespective of the L, R, W, and X bits.

If a sPMP entry matches all bytes of an access, then the L, R, W and X bits determine whether the access succeeds or fails. If the privilege mode of the access is M, the access succeeds. Otherwise, if the L bit is set or the privilege mode of the access is U, then the access succeeds only if the R, W, or X bit corresponding to the access type is set.

If no sPMP entry matches an S-mode access (i.e., there is no such a sPMP entry whose L bit is set and region contains the memory access), the access succeeds, otherwise the access is checked according to the permission bits in sPMP entry. If no sPMP entry matches an U-mode access, but at least one sPMP entry is implemented, the access fails.

9.2.5 SMAP and SMEP with sPMP

For SMAP, the SUM (permit Supervisor User Memory access) bit in the status register is leveraged to indicate the privilege with which S-mode loads, stores, and instruction fetches access physical memory.

- When SUM=0, S-Mode physical memory accesses to memory that are accessible by U-Mode (U=1 in *RV32 smpcfg<x> layout* (page 33)) will fault.
- When SUM=1, these accesses are permitted.

The SUM can take effect even when page-based virtual memory is not in effect.

For SMEP, it is not allowed for S-mode to execute codes in physical memory that are for U-mode (U=1 in *RV32 smpcfg<x> layout* (page 33)).

TEE Configuration

<xxx>_CFG_HAS_TEE is a global option to configure the implementation of TEE.

- **Nuclei RISC-V IP Products:** <https://www.nucleisys.com/product.php>
- **Nuclei Spec Documentation:** <https://nucleisys.com/download.php#spec>
- **Nuclei RISC-V Tools and Documents:** <https://nucleisys.com/download.php>
- **Nuclei Prebuilt Toolchain and IDE:** <https://nucleisys.com/download.php#tools>
- **NMSIS:** <https://github.com/Nuclei-Software/NMSIS>
- **Nuclei SDK:** <https://github.com/Nuclei-Software/nuclei-sdk>
- **Nuclei Linux SDK:** <https://github.com/Nuclei-Software/nuclei-linux-sdk>
- **Nuclei Software Organization in Github:** <https://github.com/Nuclei-Software/>
- **RISC-V MCU Organization in Github:** <https://github.com/riscv-mcu/>
- **RISC-V MCU Community Website:** <https://www.rvmcu.com/>
- **Nuclei riscv-openocd:** <https://github.com/riscv-mcu/riscv-openocd>
- **Nuclei riscv-gnu-toolchain:** <https://github.com/riscv-mcu/riscv-gnu-toolchain>